



石河子大学信息科学与技术学院 《智能计算系统》课程实验报告

实验名称:	实验三
学生姓名:	卢瑶
学 号:	20211018341
学 院:	信息科学与技术学院
专业年级:	计科 2021 级 5 班

实验三 图像识别

一、实验目的：

- 1、掌握 FASHION 数据集
- 2、掌握 Class 的编程流程
- 3、编写程序实现常用的操作

二、实验要求：

- 1、用 Tensorflow 编程框架和 Python 语言实现源程序的编写；
- 2、源程序应书写规范，适当添加注释；
- 3、实验课对源程序进行编辑、调试、运行；实验结束后尽快完成实验报告，并按规定时间上交实验报告。

三、实验内容

编写程序实现 FASHION 数据集图像识别。

四、实验结果

4.1 实验代码

```
1. import tensorflow as tf
2. from tensorflow.keras.layers import Dense, Flatten
3. from tensorflow.keras import Model
4.
5. fashion = tf.keras.datasets.fashion_mnist
6. (x_train, y_train), (x_test, y_test) = fashion.load_data()
7. x_train, x_test = x_train / 255.0, x_test / 255.0
8.
9. class MnistModel(Model): #基于"类"构造神经网络模型
10.     def __init__(self):
11.         super(MnistModel, self).__init__()
12.         self.flatten = Flatten()
13.         self.d1 = Dense(128, activation='relu')
14.         self.d2 = Dense(10, activation='softmax')
15.
16.     def call(self, x):
17.
18.         x = self.flatten(x)
19.         x = self.d1(x)
20.         y = self.d2(x)
21.         return y
22.
23.
```

```
24.model = MnistModel()
25.
26.model.compile(optimizer='adam',
27.               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
28.               metrics=['sparse_categorical_accuracy'])
29.
30.model.fit(x_train, y_train, batch_size=32, epochs=5, validation_data
           =(x_test, y_test), validation_freq=1)
31.model.summary()
```

首先，我们导入了 TensorFlow 库，这是一个广泛应用于数值计算和机器学习的强大工具库。**Dense** 和 **Flatten** 是 Keras 框架中用于构建神经网络层的类。**Dense** 层是一种全连接层，通常用于网络的输出层或内部层；**Flatten** 层则用于将多维输入一维化，以便在全连接层前处理数据。

接着，我们加载了 Fashion MNIST 数据集，并分为训练集和测试集。数据集中的每个图像都以 28x28 像素的形式存储。为了加速训练过程并改善模型的性能，将这些图像的像素值归一化到 0 到 1 之间，这是通过将每个像素值除以 255 实现的。

之后，定义了一个名为 **MnistModel** 的类，它继承自 Keras 的 **Model** 基类。这个类的构造函数中初始化了三个层：一个 **Flatten** 层和两个 **Dense** 层。**Flatten** 层将二维图像数据转换为一维数组，以便进行处理。第一个 **Dense** 层（也称为隐藏层）有 128 个神经元并使用 ReLU 激活函数，它负责学习输入数据的非线性特征。第二个 **Dense** 层（也称为输出层）有 10 个神经元，对应于 10 个类别的输出，并使用 softmax 激活函数，将输出转换为概率分布。

在类中还定义了一个 **call** 方法，它指定了数据在模型中的传递方式。数据首先通过 **Flatten** 层，然后依次通过两个 **Dense** 层，最终返回模型的输出。

创建 **MnistModel** 类的实例后，接下来通过 **compile** 方法来编译模型。在编译过程中，指定了优化器为 Adam，这是一种基于随机梯度下降的优化算法，能够自动调整学习率，适用于大多数深度学习任务。损失函数使用的是 **SparseCategoricalCrossentropy**，它用于多类分类问题，适合于标签为整数的情况。在这里，因为 **from_logits** 参数设置为 **False**，表示输出已经是概率分布形式，无需转换。最后，指定了评估模型性能的指标为 **sparse_categorical_accuracy**。

然后，使用 `fit` 方法来训练模型。在训练时，将归一化的图像和相应的标签输入模型。设置了小批量的大小为 32，总共训练 5 个时代（epochs）。此外，还通过 `validation_data` 参数提供了测试集数据，用以评估模型在未见数据上的性能，`validation_freq` 参数设置为 1 表示每个 epoch 后都进行一次验证。

最后，`summary` 方法被调用来输出模型的结构，包括每层的名称、形状和参数数量。

4.2 实验运行结果

实验结果如下所示📌：

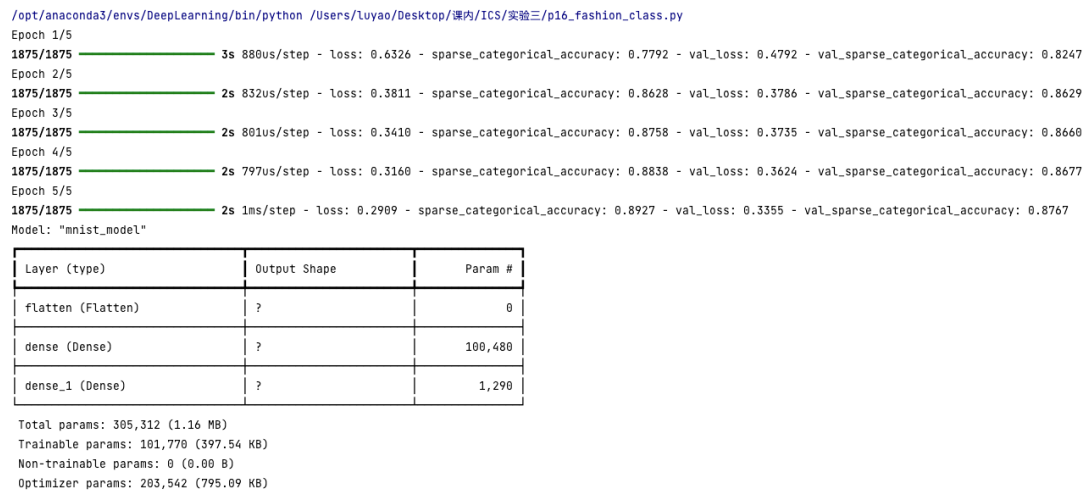


图 1 模型训练结果

1) 训练和验证过程

1. 第一个训练周期（Epoch 1）：

- 训练损失（`loss`）：0.6348，这是模型在训练集上的性能指标，表示模型在分类时平均会犯多大的错误。较高的初始损失是正常的，因为模型开始时是未训练的。
- 训练精度（`sparse_categorical_accuracy`）：77.85%，表明在训练集上，模型正确分类图像的比例。
- 验证损失（`val_loss`）：0.4393，即模型在测试集上的平均损失。通常希望这个值尽可能低。
- 验证精度（`val_sparse_categorical_accuracy`）：84.39%，即模型在测试集上的分类准确度。

2. 随着训练周期的增加 (Epoch 2-5):

- 训练损失逐渐减少, 从 0.6348 降低到 0.3028, 表明模型在识别训练数据方面越来越准确。
- 训练精度稳步提升, 最终达到 88.85%, 这意味着模型在学习过程中不断改进, 对训练图像的分类越来越准确。
- 验证损失从 0.4393 降低到 0.3560, 表明模型在测试集上的泛化能力提高。
- 验证精度从 84.39%提升到 87.09%, 表明模型在未见过的测试数据上的性能提升, 泛化能力较强。

整个训练过程在保持较低的训练和验证损失的同时逐步提升精度, 没有出现明显的过拟合现象, 即模型在测试集上的性能与训练集相似, 泛化良好。

2) 模型结构

输出的最后部分显示了模型的结构概要。模型名为"mnist_model", 由两个层组成:

- **Flatten 层:** 用于将 28x28 的二维图像展平成一维的向量。展平操作没有参数, 因此参数数量为 0。
- **Dense 层:** 第一个全连接层有 128 个神经元 (由于输入是 784 个像素点, 所以有 100,480 个参数), 第二个全连接层是输出层, 有 10 个神经元对应 10 个类别 (1,290 个参数)。

模型的总参数数为 305,312 个, 这些参数在训练过程中通过优化器调整以最小化损失函数。

3) 训练参数和硬件信息

- **训练参数 (Trainable params):** 101,770 个, 这部分参数是在训练过程中学习和调整的。
- **非训练参数 (Non-trainable params):** 0 个, 表明模型没有使用预训练层或冻结任何参数。
- **优化器参数 (Optimizer params):** 203,542 个, 这是优化器 Adam 使用的内部参数数量。

五、实验小结

5.1 数据集

Fashion MNIST 是一个现代的数据集，旨在替代传统的手写数字识别数据集 MNIST，以解决更复杂的图像识别问题。这个数据集由 Zalando（一家欧洲的时尚科技公司）发布，包含了 10 种类别的服饰图像，用于开发和测试机器学习模型在图像分类任务上的表现。

1) 数据集结构

Fashion MNIST 包含 70,000 张灰度图像，每张图像的分辨率为 28x28 像素，整个数据集被分为两部分：

1. **训练集**：包含 60,000 张图像，用于训练机器学习模型。
2. **测试集**：包含 10,000 张图像，用于评估模型的泛化能力。

类别描述

数据集中的图像被分为以下 10 个类别，每个类别都是关于服装的一种类型，各类别平均分布：

1. T-shirt/top（T 恤/上衣）
2. Trouser（裤子）
3. Pullover（套衫）
4. Dress（连衣裙）
5. Coat（外套）
6. Sandal（凉鞋）
7. Shirt（衬衫）
8. Sneaker（运动鞋）
9. Bag（包）
10. Ankle boot（踝靴）

2) 应用目的

Fashion MNIST 的设计初衷是为了提供一个比原始 MNIST 手写数字识别更具挑战性的问题，同时保持数据集处理的可接入性和管理的便捷性。图像的低分辨率意味着模型可以在不依赖于高性能计算资源的情况下进行训练和测试，使其成为教学和基础研究中非常受欢迎的选择。

3) 使用场景

Fashion MNIST 适用于各种机器学习的场景，尤其是在计算机视觉、图像识别和分类研究领域。由于其内容的多样性和实际应用的相关性，它常被用于测试不同的算法效果，包括但不限于神经网络（如卷积神经网络）、支持向量机、随机森林等。

5.2 流程图

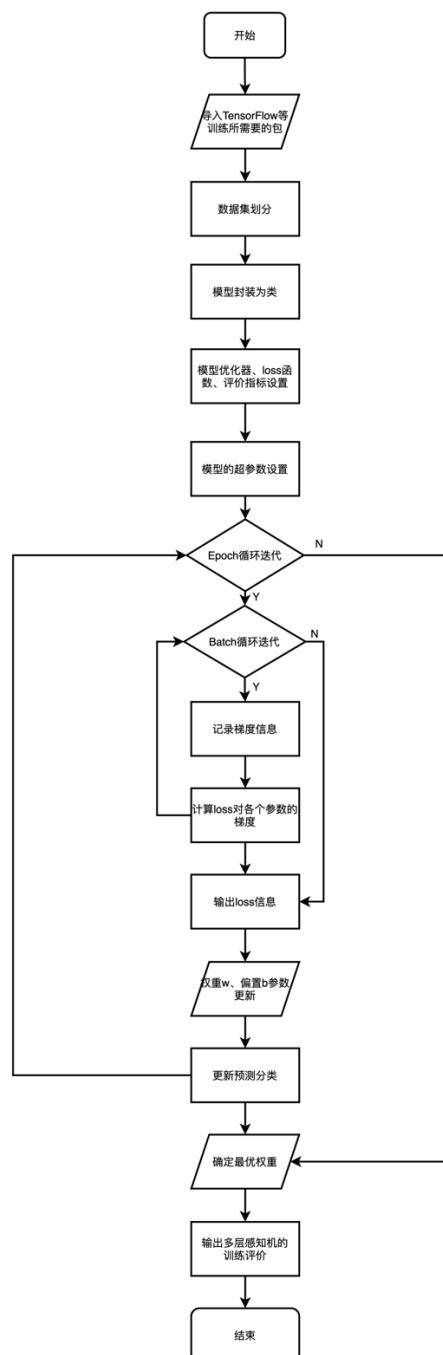


图 2 模型运行流程图

第一步，导入 TensorFlow 库和所需的模块。

第二步，使用 `tf.keras.datasets.fashion_mnist` 下载 Fashion MNIST 数据集，该数据集包含 10 类服装物品的灰度图像，每个图像大小为 28×28 像素。

第三步，将数据集划分为训练和测试集，其中 `x_train` 和 `x_test` 存储图像数据，`y_train` 和 `y_test` 存储相应的标签。

第四步，对 `x_train` 和 `x_test` 中的图像数据进行预处理，将其缩放到 0 到 1 之间的值。

第五步，定义一个继承自 Keras Model 的 `MnistModel` 类，并在构造函数中初始化模型的层。

第六步，在 `MnistModel` 类中，定义 `call` 方法以定义模型的前向传播逻辑。首先，通过 `Flatten` 层将输入的二维图像数据展平为一维，然后通过 `Dense` 层进行线性变换和激活，最后通过另一个 `Dense` 层生成模型的输出。

第七步，创建 `MnistModel` 类的实例，即实例化模型。

第八步，使用 `compile` 方法配置模型的损失函数、优化器和评估指标。这里使用 `SparseCategoricalCrossentropy` 损失函数和 `Adam` 优化器。

第九步，使用 `fit` 方法训练模型。传入训练数据、批大小、训练轮数、验证数据等参数，训练模型并在训练过程中对其进行评估。

第十步，使用 `summary` 方法打印模型的概述信息，包括每个层的名称、类型、输出形状和参数数量。

5.3 多层神经网络结构图

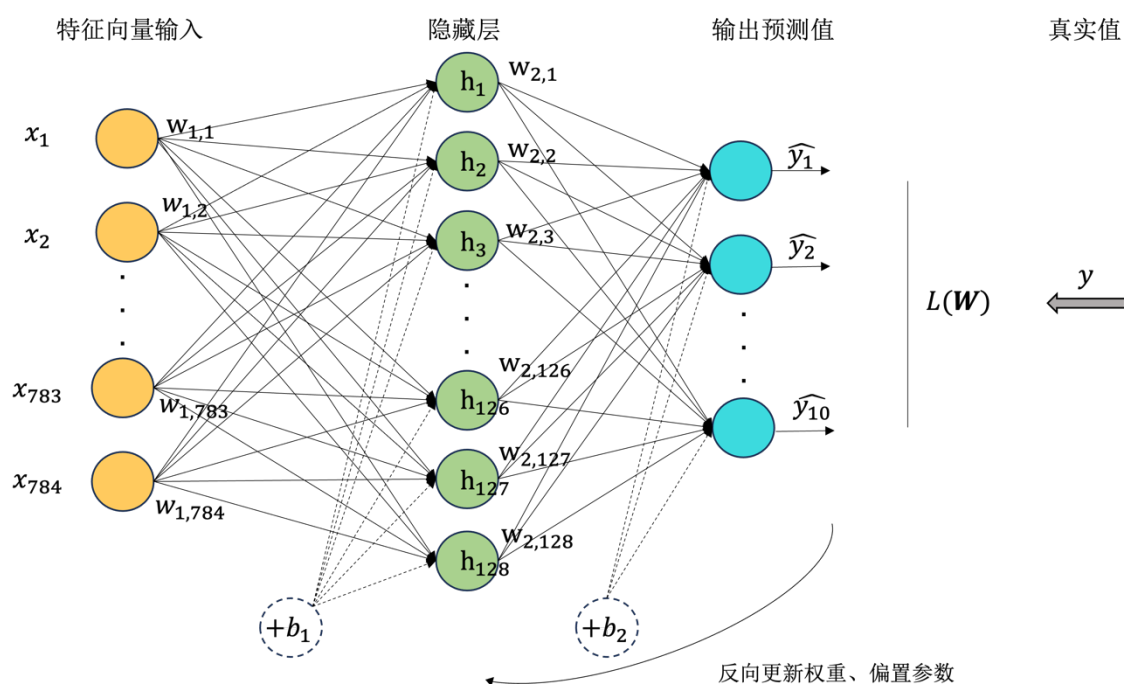


图 3 多层感知机网络结构图

从上图我们可以看到：

1. 输入层 (Input Layer):

- 输入层由图像像素组成，每个像素对应一个输入节点。在这个图像中，输入层包含了 $28 \times 28 = 784$ 个节点，表示一个 28×28 像素的手写数字图像。

2. 隐藏层 (Hidden Layers):

- 图中显示了两个隐藏层，每个隐藏层包含多个节点。
- 每个节点通过连接权重与前一层的所有节点相连，并通过激活函数进行非线性转换。

3. 输出层 (Output Layer):

- 输出层包含了 10 个节点，每个节点表示一个数字类别 (0 到 9) 的概率。
- 输出层的激活函数通常是 softmax 函数，用于将输出转换为概率分布。

4. 连接权重 (Connection Weights):

- 每个节点之间的连接都有一个权重，这些权重决定了信号在网络中传播时的重要性。
- 权重在训练过程中通过反向传播算法进行调整，以最小化损失函数并提

高模型的性能。

5. 损失函数 (Loss Function):

- 损失函数用于衡量模型的预测与实际标签之间的差异。
- 在训练期间，模型试图最小化损失函数，以使预测结果尽可能接近真实标签。

6. 优化器 (Optimizer):

- 优化器是用于调整连接权重的算法，以最小化损失函数。
- 常见的优化算法包括随机梯度下降 (SGD) 和 Adam 优化器。

与前文实验不同的是，我们将模型的全连接层、前向传播封装到了类中，提高了代码可读性的同时，整体架构也更加的规整、简洁。

六、成绩